

BUIDL CTC 2026 FALL

Remit-to-Own

A relative abroad pays on Ethereum.
The motorcycle here still starts.

THE SITUATION

Two ordinary things that do not fit together

- ▶ Buying a motorcycle or a solar panel on installments is normal across much of the world.
- ▶ Having a relative abroad cover those payments is just as normal.
- ▶ But the seller is at home and the money is abroad, and neither side can see the other's ledger.
- ▶ So the gap gets filled by remittance operators and local agents, who each take a cut and each have to be trusted.

TODAY

Nobody can check

The shop cannot see the transfer. The family cannot see the account. Somebody in the middle tells both sides what happened, and charges for it.

THE IDEA

Let the payment prove itself.

Each plan gets its own collection address on Ethereum.
Attestcoin proves the transfer to Creditcoin, and the device answers to that proof alone.

HOW IT BEHAVES

Pay more, it works longer. Stop, it locks.

PAY

Prepaid days, by the second

Send a third of an installment or four at once. Service extends in proportion, so small sums are never rounded away.

STOP

It simply locks

No default, no collections, no chasing. The device locks, which is how pay-as-you-go financing already works in the field.

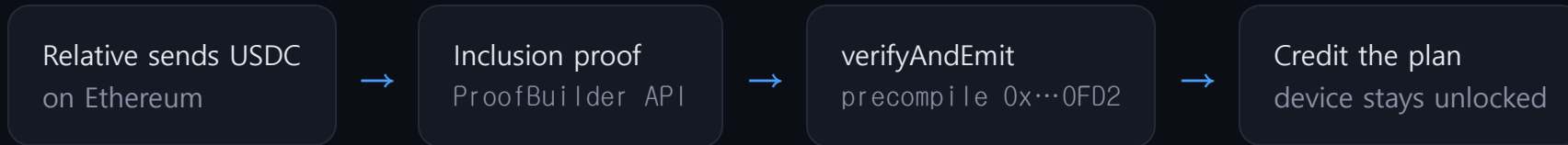
FINISH

It becomes yours

When the price is covered, ownership transfers and it never locks again.

Nothing is lent, so nothing has to be recovered. That is what keeps this design standing where cross-chain credit ideas fall over.

A contract cannot read another chain



- ▶ The transfer is public. Anyone can open Etherscan and read it. **A contract on Creditcoin cannot.** It has no eyes outside its own chain and knows only what somebody hands it.
- ▶ So it either trusts the messenger or verifies the proof. An invalid proof reverts the whole call, and a replay guard means one payment can never count twice.
- ▶ Anyone can submit a proof, and a relay can stall but cannot lie.

"I have been making payments and yet am told that I have not paid. The whole village is complaining."

Off-grid solar customer, 60 Decibels 2024. She could see her own payments. The company's ledger was the one that counted. Here the proof moves the ledger, so nobody has to be persuaded.

IT ALREADY WORKS, ON CC3 TESTNET

Both source chains, doing different jobs

PLAN	PAYMENTS PROVEN	STATE
Solar lantern, 12 USDC Sepolia	3.00, ours end to end	RUNNING 30 days bought
Delivery truck, 80,000 USDC	342.59, 1,500.00, 599.68	RUNNING 35 days bought
Delivery truck, 50,000 USDC	23.48, 1,490.71, 97.47, 107.41, 1,946.58	RUNNING 53 days bought
Motorcycle, 1,800 USDC	89.42, then the balance	OWNED paid off
Solar home system, 800 USDC	one transfer, capped at the balance	OWNED paid off

The Sepolia plan is ours end to end, from the collection address to the payment, so a relative sending money and a device switching on can be shown on demand. The mainnet plans point at busy addresses nobody here controls, which put the proof path in front of live traffic at no cost and is what surfaced the bug that killed four proofs in six: payment transactions are often swaps carrying several tokens, which the first version rejected outright. Neither chain alone would have done both jobs.

An adversarial audit, and what it found

- ▶ **Two critical holes.** Naming an address you do not control as your collector, and any accepted token paying any plan. Both closed.
- ▶ **A bug only live data could find.** Real payment transactions are often swaps carrying a second token, which the first version rejected outright. Four of six real payments died before the fix.
- ▶ **38 tests, all passing.** 23 on behaviour, 15 regression tests, one per attack.
- ▶ The precompile is never mocked. It is exercised against real Ethereum transactions.

THE LESSON

A proof shows what happened, not who owns what

Attestcoin proves a transfer took place. It cannot prove whose address received it. Treating the second as though it followed from the first was the critical bug, and the fix is a restriction, not a cleverer proof.

Where the money is, and where the ledger can afford to be

< 1¢

to prove one payment on Creditcoin, measured. The same record on Ethereum costs dollars, which is more than a single instalment is worth on an 800 USDC solar plan.

This is not a saving. Below this price the product does not exist.

- ▶ Ethereum is where the stablecoins are. Creditcoin is where a ledger can be rewritten on every small payment without eating the payment.
- ▶ Attestcoin is the only thing that lets those two facts hold at once.
- ▶ The merchant needs no crypto knowledge. They read one boolean, `isActive`.
- ▶ Every proven payment leaves a repayment record for a household that had none. Whether a lender ever prices that record is not something this project can claim yet.
- ▶ \$700B reached low and middle income countries as remittances in 2024. World Bank.

What is built, and what is not

BUILT AND VERIFIED

The full loop

Contract on CC3, proof relay, and a device page that watches the chain and alerts the moment a payment is proven, in four languages. Proven end to end against live mainnet transfers, with an adversarial audit and a regression suite behind it.

- ▶ The device lock is on-chain state. Wiring it to a real controller is a hardware integration, not this submission.
- ▶ Attestation lags the source chain by about nine minutes by design. For monthly financing that is immaterial.
- ▶ **Plans are opened by the operator**, because a proof shows that a transfer happened and nothing on chain shows who an address belongs to. That is a deliberate restriction, not an oversight: left open, the same gap lets anyone claim a busy address as their collector and bank a stranger's deposit.
- ▶ The roadmap closes it with Attestcoin itself. An address proves it is yours by sending a marked transfer, which is proven the same way a payment is, so registration needs no operator and no permission.

Send money. Keep it working.

Remit-to-Own. Device financing that answers to a proof instead of a middleman.

RemitToOwn · 0x59FEF8771Da4248b89F7D6052b9d10fDfb13D223 · Creditcoin CC3